Programação Funcional

Prática a05 - Recursão II

Ficha PL2 — Até tarefa 5
Cap. 5, especialmente pp. 39 – 43

Revisão rápida de pattern matching

Funções com várias cláusulas

```
f :: (Int,Char,Int) -> Int

f (y,'a',x) = y + x
f (z,'b',x) = z * x
f (x,y,z)   = x
```

Quando chamamos f, Haskell **procura de cima para baixo** a primeira cláusula **cujo padrão corresponde ao argumento**.

A ordem interessa?

```haskell
f :: (Int,Char,Int) -> Int

f (y,'a',x) = y + x
f (z,'b',x) = z * x
f (x,y,z)   = x
```

- f (3,'a',5)
- f (9,'B',0)
- f (5,'b',4)

- Qual cláusula é testada primeiro?
- Qual padrão corresponde?
- O que acontece se trocar a ordem das cláusulas?

A ordem das condições interessa!

Resolver:

f (3,'a',5)

f (9,'B',0)

E se trocarmos a primeira condição pela última?

```
f :: (Int,Char,Int) -> Int


f (y,'a',x) = y + x
f (z,'b',x) = z * x
f (x,y,z)   = x
```

if/then/else vs pattern matching

A função opp:

```
opp :: (Int,(Int,Int)) -> Int

opp z =
    if fst z == 1
        then fst (snd z) + snd (snd z)
        else if fst z == 2
            then fst (snd z) - snd (snd z)
            else 0
```

opp (1,(2,3))

operação; valores

Qual o resultado de opp (1,(2,3))?

opp com pattern matching

```haskell
opp' :: (Int,(Int,Int)) -> Int

opp' (1,(x,y)) = x + y
opp' (2,(x,y)) = x - y
opp' _         = 0
```

Pattern matching

```haskell
opp :: (Int,(Int,Int)) -> Int

opp z =
    if fst z == 1
        then fst (snd z) + snd (snd z)
        else if fst z == 2
            then fst (snd z) - snd (snd z)
            else 0
```

if/then/else conditions

Qual versão é mais fácil de ler?

opp com guards

Relembrar:

- | → introduz uma guarda
- condição → tem de produzir Bool
- otherwise → caso contrário

```haskell
oppGuardas :: (Int,(Int,Int)) -> Int
oppGuardas (op,(x,y))
    | op == 1   = x + y
    | op == 2   = x - y
    | otherwise = 0
```

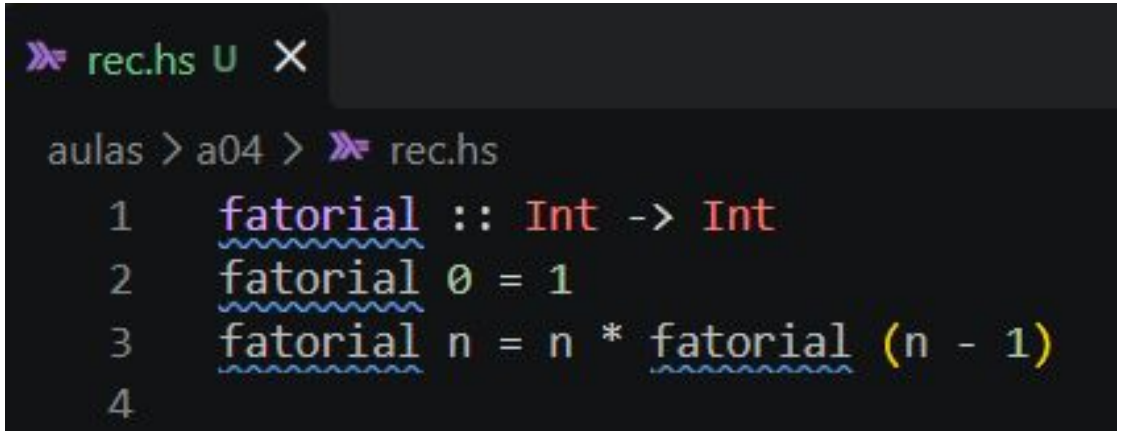
```haskell
opp :: (Int,(Int,Int)) -> Int

opp z =
    if fst z == 1
        then fst (snd z) + snd (snd z)
        else if fst z == 2
            then fst (snd z) - snd (snd z)
            else 0
```

if/then/else conditions

Fatorial que nunca termina?

O que acontece com fatorial (-2)?



Função fatorial completa

```haskell
fact :: Int -> Int
fact n
    | n < 0      = error "factorial indefinido para negativos"
    | n == 0     = 1
    | otherwise = n * fact (n - 1)
```

A função agora trata explicitamente todas as categorias possíveis de Int

Recursão sobre listas

Função reverse que recebe uma lista e retorna a lista reversa.

```haskell
reverse :: [a] -> [a]

reverse [] = []
reverse (x:xs) = reverse xs ++ [x]
```

Caso base:
[] → []

Caso recursivo:
1. inverter xs
2. colocar x no fim do resultado

reverse [1,2,3]

→ reverse [2,3] ++ [1]

→ (reverse [3] ++ [2]) ++ [1]

→ ((reverse [] ++ [3]) ++ [2]) ++ [1]

→ (([] ++ [3]) ++ [2]) ++ [1]

→ [3,2,1]

Execução demorada

reverse [1,2,3]

→ reverse [2,3] ++ [1]

→ (reverse [3] ++ [2]) ++ [1]

→ ((reverse [] ++ [3]) ++ [2]) ++ [1]

→ (([] ++ [3]) ++ [2]) ++ [1]

→ [3,2,1]

A cada chamada recursiva, aparece um operador ++ que fica pendente: só é executado depois de todas as chamadas de reverse serem calculadas.

reverse [lista com 500 elementos]: teríamos de esperar por 500 reverse e só depois podíamos fazer todos os ++ um de cada vez.

Este processo é demorado e computacionalmente ineficiente: $O(n^2)$

Acomuladores

E se em vez de esperar pelo regresso da recursão para construir o resultado, o construirmos enquanto avançamos?

Lista por tratar	Resultado já construído
[1,2,3]	[]
↓	
[2,3]	[1]
↓	
[3]	[2,1]
↓	
[]	[3,2,1]

```haskell
reverseAcc :: [a] -> [a]
reverseAcc [] = []
reverseAcc lista = aux lista []
  where
    aux [] acumulado = acumulado
    aux (x:xs) acumulado = aux xs (x:acumulado)
```

Um acumulador é um argumento adicional que transporta o resultado parcial de uma chamada recursiva para a seguinte.

reverseAcc

```haskell
reverseAcc :: [a] -> [a]
reverseAcc [] = []                    -- reverse de uma lista vazia é lista vazia
reverseAcc lista = aux lista []       -- caso contrario usamos a função auxiliar aux
  where
    aux [] acumulado = acumulado                    -- caso base
    aux (x:xs) acumulado = aux xs (x:acumulado)     -- caso recursivo
```

Começamos com a lista original e um acumulador vazio [ ]

Caso base: aux [ ] acumulado = acumulado — Quando já não existem elementos para tratar, o acumulador contém o resultado final.

Caso recursivo: aux (x:xs) acumulado = aux xs (x:acumulado) — Retiramos x da lista original e colocamo-lo à frente do acumulador.

reverseAcc [1,2,3]

```haskell
reverseAcc :: [a] -> [a]
reverseAcc [] = []
reverseAcc lista = aux lista []
  where
    aux [] acumulado = acumulado
    aux (x:xs) acumulado = aux xs (x:acumulado)
```

chamada	lista por tratar	acc
aux [1,2,3] []	[1,2,3]	[]
aux [2,3] [1]	[2,3]	[1]
aux [3] [2,1]	[3]	[2,1]
aux [] [3,2,1]	[]	[3,2,1]

aux [1,2,3] []

→ aux [2,3] [1]

→ aux [3] [2,1]

→ aux [] [3,2,1]

→ [3,2,1]

reverseAcc

Em cada chamada, o que diminui?: a lista por tratar

O que cresce?: o acumulador

Quando chegamos ao caso base, ainda temos trabalho para fazer?: Não. O resultado já está no acumulador.

reverceAcc constrói o resultado à medida que passa pela lista. Trata cada elemento diretamente e mantém o resultado parcial.

Complexidade computacional: $O(n)$

Soma com acumulador: somaAcc

```haskell
somaAcc :: [Int] -> Int
somaAcc xs = aux xs 0
  where
    aux [] acc = ...
    aux (x:xs) acc = ...
```

O que deverá conter acc?

Qual deve ser o resultado quando a lista ficar vazia?

O que devemos passar como novo acumulador?

somaAcc

```haskell
somaAcc :: [Int] -> Int
somaAcc xs = aux xs 0
  where
    aux [] acc = acc
    aux (x:xs) acc = aux xs (acc + x)
```

Função principal: somaAcc xs = aux xs 0

começa a processar a lista xs com um acumulador igual a 0.

Caso base: aux [] acc = acc

Caso recursivo: aux (x:xs) acc = aux xs (acc + x)

somaAcc [1,2,3]

```haskell
somaAcc :: [Int] -> Int
somaAcc xs = aux xs 0
  where
    aux [] acc = acc
    aux (x:xs) acc = aux xs (acc + x)
```

somaAcc [1,2,3]

→ aux [1,2,3] 0

→ aux [2,3] 1

→ aux [3] 3

→ aux [] 6

→ 6

Exercícios

Num ficheiro acc.hs, implementar e testar as seguintes funções:

- pertence — Função que recebe um valor e uma lista e devolve True se esse valor pertencer à lista e False caso contrário.
- removePrimeiro — Função que recebe um valor e uma lista e devolve a lista após remover apenas a primeira ocorrência desse valor.
- contaOcorrencias — Função que recebe um valor e uma lista e devolve o número de vezes que esse valor aparece na lista.

Para cada função:

- 1. escrever a assinatura;
- 2. identificar o caso base;
- 3. identificar o caso recursivo;
- 4. testar no GHCi.

```haskell
contaOcorrencias :: [Int] -> Int
-- contaOcorrencias recebe uma lista de Int e devolve um Int,
-- correspondente ao número de vezes que um valor aparece na lista.

contaOcorrencias [] = 0
-- Caso base: numa lista vazia existem 0 ocorrências.

contaOcorrencias (x:xs)
    | x == valor = 1 + contaOcorrencias xs
      -- Se o primeiro elemento x for igual ao valor procurado,
      -- contamos 1 ocorrência
      -- e continuamos a procurar no resto da lista xs.
    | otherwise  = contaOcorrencias xs
      -- Caso contrário, não contamos nada
      -- e continuamos a procurar no resto da lista.
```

Ficha PL2 - Até tarefa 4

no git https://github.com/v2denny/pf2627

pasta >aulas>a05>FichaPL2.pdf

Desafio

Implementa a função quicksort, sem usar sort.

A função recebe uma lista de elementos ordenáveis e devolve essa lista ordenada por ordem crescente.

quicksort :: Ord a => [a] -> [a]

Exemplos para testar:

```
quicksort [3,1,4,2]
-- [1,2,3,4]

quicksort [5,2,8,1,5]
-- [1,2,5,5,8]

quicksort []
-- []

quicksort [7]
-- [7]
```